



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D1: Linux开发环境

Linux下Web Server的设计

讲授内容

- 训练目的
- 训练原理
- 训练内容



Linux开发环境

- 训练目的
 - 熟悉基于Linux内核的操作系统ubuntu
 - 熟悉基于GNU工具链的开发环境
 - ✓ ubuntu (bourne again shell)
 - ✓ gcc
 - ✓ gdb
 - ✓ make
 - 掌握Linux程序编译调试方法
 - 掌握makefile的编写规则



训练原理：ubuntu的安装

- Ubuntu是一个以桌面应用为主的Linux操作系统。Ubuntu基于Debian发行版和Gnome/Unity桌面环境。
- Ubuntu桌面版下载地址：
<https://cn.ubuntu.com/download/desktop>



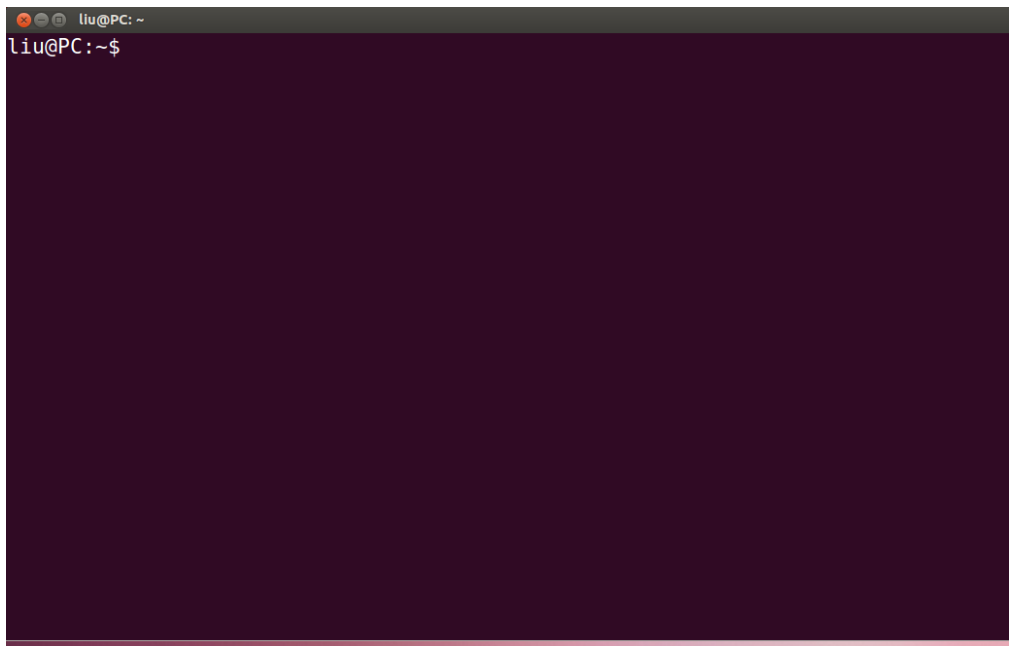
训练原理：ubuntu的安装

- 三种安装方式
 - 下载ubuntu安装镜像文件，在PC或笔记本的Windows系统中安装虚拟机软件（例如VMware workstation, VirtualBox），在虚拟机软件中安装ubuntu）
 - 制作ubuntu安装镜像光盘或U盘，引导启动后在PC或笔记本上安装ubuntu（和Windows形成双系统）
 - 制作ubuntu安装镜像光盘或U盘，引导启动后在PC或笔记本上安装ubuntu（卸载Windows）



训练原理：Linux的操作

- 在开始菜单中找到terminal或者快捷键Ctrl+Alt+t，打开控制台程序



Linux常用命令及工具

- ✓ cp 源文件 目标文件： 拷源文件成为目标文件
- ✓ cp -r 源目录 目标目录： 连子目录一起，拷源目录成为目标目录
- ✓ man 命令： 查看该命令的帮助文件
- ✓ su 用户名： 转变用户身份
- ✓ sudo -i： 转变成root用户（超级用户）
- ✓ pwd： 列出用户当前所在目录
- ✓ cd： 转变目录 **cd .. 返回上一级目录**
- ✓ **ls： 列文件及目录**
- ✓ mkdir 目录名： 建立目录
- ✓ rmdir 目录名： 删除空目录
- ✓ rm 文件名： 删除文件
- ✓ rm -r 目录名： 删除非空目录
- ✓ **gedit 文件名： 编辑文本文件内容**

```
root@ubuntu:/home/zhangsan# cd ..  
root@ubuntu:/home#
```

Linux常用命令

- mv 文件名 文件名： 文件改名
- mv 文件名 目录名： 文件移动到目标目录
- mv 目录名 目录名： 目标目录已存在，则将源目录移动到目标目录
- cat 文件名： 查看文本文件内容
- clear 清除屏幕
- find / -name 文件名： 查找指定名字的文件
- ifconfig： 查看全部网卡配置
- ifconfig eth0 ： 查看指定网卡配置
- ifconfig eth0 192.168.0.100： 设置网卡eth0 地址为192.168.0.100
- shutdown now： 关闭系统
- reboot： 重启系统

使用Linux命令的注意事项和小技巧

- 命令和后面参数之间有空格、参数列表之间也有空格。下图中cp、sudo、rm是命令，空格后面的是参数

```
cp /root/zyy/test.c /root/zhangsan  
sudo -i  
rm -r
```

- 输入命令或路径的时候可以用键盘Tab键补全命令。例如ifconfig命令，在输入ifc后按Tab键可以补全命令

按Tab



- 在控制台中按上、下方向键可以查找历史输入记录

使用Linux命令的常见错误

- 命令和参数之间不加空格

```
root@ubuntu:~/zhangsan# sudo-i
sudo-i: command not found
root@ubuntu:~/zhangsan#
```

```
root@ubuntu:~/zhangsan# cd/root
-bash: cd/root: No such file or directory
root@ubuntu:~/zhangsan#
```

- 复制的时候源文件路径和目标路径之间不加空格

```
root@ubuntu:~/zhangsan# cp /root/zhangsan/test/root/nfs/
cp: missing destination file operand after `/root/zhangsan/test/root/nfs/'
Try `cp --help' for more information.
root@ubuntu:~/zhangsan#
```

- 配置IP地址的时候把eth0 误操作成etho

```
root@ubuntu:~/zhangsan# ifconfig etho 192.168.1.2
SIOCSIFADDR: No such device
etho: ERROR while getting interface flags: No such device
root@ubuntu:~/zhangsan#
```

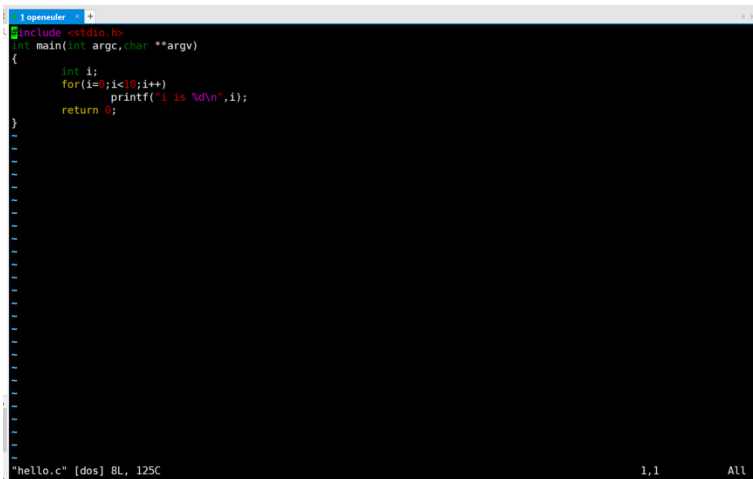
使用Linux命令的常见错误

- 当前目录下没有的文件“无中生有”，报错：找不到文件或目录
(用ls命令查看当前目录下所有文件)

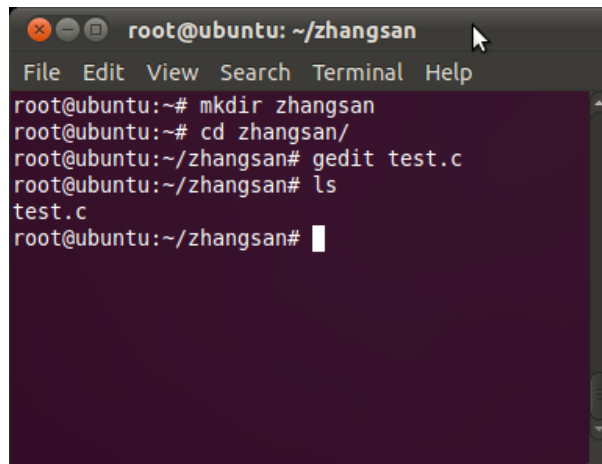
```
root@ubuntu:~/zhangsan# ls
arm-linux-gcc-4.3.2.tgz test test.c usr
root@ubuntu:~/zhangsan# cp hello /root
cp: cannot stat `hello': No such file or directory
root@ubuntu:~/zhangsan#
```

Ubuntu的使用-通过系统工具编辑C程序

- ◆ 使用gedit编辑源程序（假设命名为test.c）
在控制台中输入：gedit test.c（源程序必须以.c为后缀，这样编译器才知道是c程序）
- ◆ 在图形化界面中编辑，保存并退出gedit



```
#include <stdio.h>
int main(int argc, char **argv)
{
    int i;
    for(i=0;i<10;i++)
        printf("i is %d\n",i);
    return 0;
}
```



```
root@ubuntu: ~/zhangsan
File Edit View Search Terminal Help
root@ubuntu:~# mkdir zhangsan
root@ubuntu:~# cd zhangsan/
root@ubuntu:~/zhangsan# gedit test.c
root@ubuntu:~/zhangsan# ls
test.c
root@ubuntu:~/zhangsan#
```

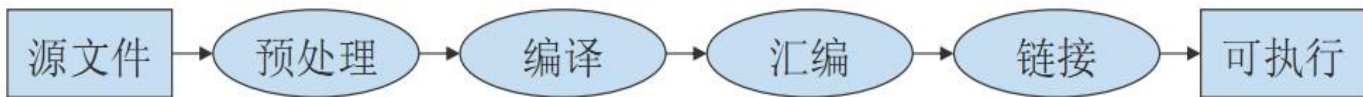
- ◆ 使用vim编辑源程序（假设命名为test.c）
在控制台中输入：vim test.c
- ◆ 在字符界面中编辑，通过命令保存并退出vim

训练原理：GCC概述

- GCC是GNU计划的一个项目。是一个自由编译器，如今的GCC已经是一个包含众多语言的编译器了（C，C++，Ada，Object C，Java及Go等）。所以，GCC也由原来的GNU C Compiler变为GNU Compiler Collection
- GCC主要包括：
 - cpp(预处理器)
 - gcc(c编译器)、g++(c++编译器)等编译器
 - binutils等二进制工具。
 - ✓ as(汇编器)
 - ✓ ld(链接器)
 - ✓ ...



训练原理：GCC编译过程



```
# cpp -o hello.i hello.c
```

```
# cc1 -o hello.s hello.i
```

```
# as -o hello.o hello.s
```

```
# ld -o hello hello.o
```



训练原理：GCC基础使用方法

- gcc命令格式：gcc [选项] <文件名>
 - -o filename：指定输出文件为filename.该选项不在乎gcc产生什么输出，无论是可执行文件，目标文件，汇编文件还是预处理后的C代码
 - 如果没有使用“-o”选项,默认的输出结果是:可执行文件为“a.out”，编译后产生的目标文件是“sourcename.o”，汇编文件是“sourcename.s”，而预处理后的C源代码送往标准输出
- 对于源代码main.c，可以通过如下命令编译成最终可执行文件（默认包含了预处理、编译、汇编及链接四个阶段）：
`gcc main.c -o main`



训练原理：GCC基础使用方法

```
#include "myprj.h"
//hello
int main(int argc, char *argv[])
{
    addRecord();
    modifyRecord();
    deleteRecord();
    exit(0);
}
```

```
//myprj.h
extern void addRecord();
extern void modifyRecord();
extern void deleteRecord();
```

```
//add.c
#include <stdio.h>
void addRecord()
{
    printf("record added successful\n");
}
```

```
//delete.c
#include <stdio.h>
void deleteRecord()
{
    printf("record deleted successfully\n");
}
```

```
//modify.c
#include <stdio.h>
void modifyRecord()
{
    printf("record modified successfully\n");
}
```

- 编译方法：首先需要生成目标文件 example.o add.o modify.o delete.o

```
gcc -c example.c
gcc -c add.c
gcc -c modify.c
gcc -c delete.c
```
- 链接 4 个目标文件，生成可执行文件 example

```
gcc example.o add.o modify.o delete.o -o example
```



训练原理：通过选项控制gcc编译阶段

-E：只激活预处理，即将预处理全部展开，该参数编译源程序后不生成文件，需要把它重定向到一个输出文件里查看结果。

```
[root@localhost linuxkc]# gcc -E example.c >result.txt
[root@localhost linuxkc]# ls
a.out example1_1 example.c example.o example.s result.txt
[root@localhost linuxkc]# more result.txt _
```

-S：只激活预处理和编译，即把文件编译成为汇编代码。

```
[root@localhost linuxkc]# gcc -S example.c
[root@localhost linuxkc]# ls
a.out example1_1 example.c example.o example.s
[root@localhost linuxkc]# _
```

-c：只激活预处理，编译和汇编，即生成目标（obj）文件。

```
[root@localhost linuxkc]# gcc -c example.c
[root@localhost linuxkc]# ls
a.out example1_1 example.c example.o
[root@localhost linuxkc]# _
```

训练原理：GCC常用编译选项

- 调试选项
 - **-g**: 以操作系统的本地格式(stabs, COFF, XCOFF,或DWARF).产生调试信息. GDB能够使用这些调试信息.
 - **-ggdb**: ggdb选项可以插入的更为丰富的调试信息, 但是生成的可执行文件有可能无法用其它调试器调试
- 调试选项的启动会让二进制文件的大小急剧增长



训练原理：GCC常用编译选项

- 优化选项

- -O 或者 -O1 优化选项：编译器会试图减少目标码的大小和执行时间
- -O2：除了涉及空间和速度交换的优化选项,执行几乎所有的优化工作，但不进行循环展开和函数内嵌
- -O3：乱序执行，循环展开的优化
- 调试时不能用优化选项，否则变量值和源代码无法对应



训练原理：GDB概述

- GDB是GNU计划开发的程序调试工具
- GDB可以完成以下四个方面的功能：
 - 启动程序，可以按照自定义的要求随心所欲的运行程序
 - 可让被调试的程序在所指定的调置的断点处停住（断点可以是条件表达式）
 - 当程序被停住时，可以检查此时程序中所发生的情况
 - 动态的改变程序的执行环境。



训练原理：GDB的启动

- 直接在shell中运行gdb命令，进入gdb界面后用 file program 装载程序。
- 在shell中启动gdb并加载可执行文件

gdb <program>

- 用gdb同时调试一个运行程序和core文件（core是程序非法执行后core dump后产生的文件）

gdb program core

- 调试正在运行的进程

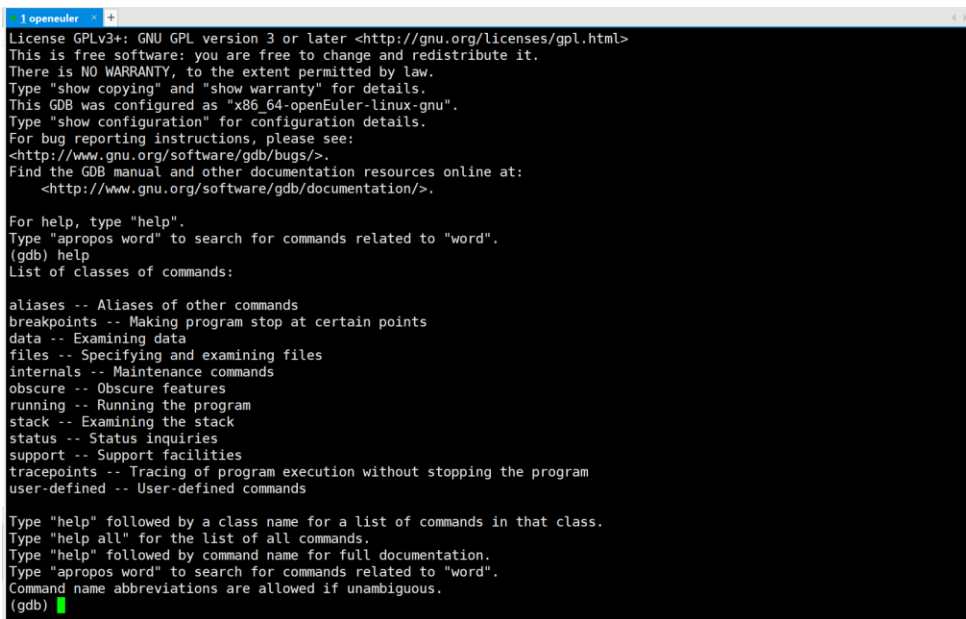
gdb program <processid>

进入gdb后用 attach <processid> 调试正在运行的进程



训练原理：GDB常用命令

- `help [command]` 在gdb中查询command的用法,如果不加command, 则列出gdb的命令分类。可以用 `help` 分类名查看此分类下所有的gdb命令
- `quit`或者`ctrl+d` 退出gdb



```
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-openEuler-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) help
List of classes of commands:

aliases -- Aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without stopping the program
user-defined -- User-defined commands

Type "help" followed by a class name for a list of commands in that class.
Type "help all" for the list of all commands.
Type "help" followed by command name for full documentation.
Type "apropos word" to search for commands related to "word".
Command name abbreviations are allowed if unambiguous.
(gdb)
```



训练原理：GDB常用命令break

功能：断点设置命令break(缩写 b)，当gdb执行到该断点时会让程序暂停运行。此时程序员可以查看运行中程序的情况。

- 格式：break [LOCATION] [thread THREADNUM] [if CONDITION]
- [LOCATION]:
 - linenum (行号)
 - filename:function
 - function name(函数名)
 - class:function (c++)
 - filename:linenum

b 123 b main b increase:main b increase:123
- [thread THREADNUM] 调试多线程程序时，切换到哪个线程或者在那个线程中设置断点。 break frik.c:13 thread 28
- [if CONDITION]: 当条件满足时，断点才生效。一般称为条件断点。CONDITION跟C语言一样

b 123 if index==2

当index为2时，程序在123行停下。



训练原理：GDB常用命令watchpoint

- watchpoint称为观察点，当观察对象的值有变化时，程序立即停止执行。
- watch <expr>：为表达式（变量）expr设置一个观察点。一旦表达式值有变化时，马上停住程序。
- rwatch <expr>：当表达式（变量）expr被读时，停住程序。
- awatch <expr>：当表达式（变量）的值被读或被写时，停住程序。
- info watchpoints：列出当前所设置了的所有观察点。



训练原理：GDB常用命令清除禁止断点或观察点

- `clear [linenum] [function name]` 清除所有断点,不会清除 watchpoints。
- `delete <num>` 清除编号为num的断点或者watchpoint。
- `disable <num>` 禁止某个断点。
- `enable <num>` 开启某个断点。



训练原理：GDB常用命令查看运行时数据

- 在调试程序时，当程序被停住时，可以使用print命令（简写命令为p），或是同义命令inspect来查看当前程序的运行数据
- print命令的格式是：print <expr> 或 print /<f> <expr>
 - <expr>是表达式，是你所调试的程序的语言的表达式（GDB可以调试多种编程语言），<f>是输出的格式，比如，如果要把表达式按16进制的格式输出，那么就是/x。
- ① 查看变量i的值：(gdb)p i
- ② 查看变量i乘5后的值：(gdb)p i*5
- ③ 查看结构体s的成员变量name的值：(gdb)p s.name
- 可以设置一些自动显示的变量，当程序停住时，或是在单步跟踪时，这些变量会自动显示，指令为display，与print类似，其格式为：display <expr> 或 display/<fmt> <expr> 或 display /<fmt> <addr>
- 其它相关指令：undisplay, delete display, disable display, enable display, info display



训练原理：GDB常用命令查看源代码

- 在使用GDB调试的过程中，可以用list命令来打印程序的源代码（简写l）。list不带参数时一般是打印当前行的上5行和下5行，如果显示函数是上2行下8行，默认是10行。在list后面可以跟以下这些参数：
 - <linenum> 行号。
 - <+offset> 当前行号的正偏移量。
 - <-offset> 当前行号的负偏移量。
 - <filename:linenum> 哪个文件的哪一行。
 - <filename:function> 哪个文件中的哪个函数。
 - <*address> 程序运行时的语句在内存中的地址。
- GDB还提供了源代码搜索的命令：
 - forward-search <regexp> 和 search <regexp>：向前面搜索
 - reverse-search <regexp>：全部搜索。
 - 其中，<regexp>就是正则表达式，也是一个字符串的匹配模式。



训练原理：GDB调试命令

- step 单步调试命令，一次执行一行程序。
- next 单步调试命令，但跳过函数调用。
- finish 单步调试时直接从一个函数中返回
- disassemble 显示汇编代码。



训练原理：GDB调试命令

- backtrace或者bt 查看目前程序的堆栈情况。
- whre查看目前程序的堆栈情况。
- up/down 向上或者向下移动一个堆栈。
- frame<num>或者f 移动到第num个堆栈。
- 当移动到某个堆栈时，便可以用gdb命令查看在那个堆栈中的局部变量。



训练原理：GDB常用命令

- `dir` 源代码路径：gdb默认有`$cdir`和`$cwd`两个源代码搜索路径，如果你要调试程序的源代码不在这两个路径中，可用`dir`命令增加。
- `r [[参数1]...[参数n]]`：在gdb中运行已经装载的可执行文件，参数为程序所需的参数。
- `info [**]` 查看设置的断点，函数名称，类名等。
`info b` 查看已经设置的断点名称



训练一：gcc+gdb训练

- 训练内容
 - 在Linux下通过C语言程序设计应用程序，其功能主要包括：
通过定义单向链表的数据结构，设计创建链表、插入结点、遍历结点等基本算法
 - 在Linux下通过GNU工具链编译、调试、运行该应用程序
- 训练报告特别要求：需要将编译与调试的过程截图



训练内容： 参考的代码片段

- 预定义数据结构

```
typedef struct stuInfo {  
    char  stuName[10];    /*学生姓名*/  
    int   Age /*年龄*/  
} ElemType  
typedef struct node {  
    Elemtype data;  
    struct node *next;  
} ListNode, *ListPtr;
```



训练内容： 参考的代码片段

```
int main(int argc,char argv[])
{
    while(1)
    {
        printf("1 Create List\n");
        printf("2 Printf List\n");
        printf("3 Insert List\n");
        printf("4 Quit\n");
        char command =getchar();
        switch(command)
        {
            case '1':ListHead = CreateList();|
            break;
            case '2':PrintList(ListHead);
            break;
            case '3':InsertList(ListHead);
            break;
            case '4':
            return 0;
        }
    }
    return 0;
}
```



训练原理：make与Makefile

- 实际的Linux程序开发往往涉及多个模块，根据习惯通常将不同模块的实现代码放在不同的源文件中
- 以下几个文件示例是一个简单的多模块Linux程序addressbook，其中包含4个C语言源文件和一个C语言头文件，实现了通信录的功能



训练原理：make与Makefile

main.c 文件示例如下。↵

```
#include "addressbook.h"↵
int main(int argc, char **argv)↵
{    ↵
    for(;;){↵
        scanf("%d", &cmd);↵
        switch(cmd){↵
            case 0: ↵
                find_rec();↵
                break;↵
            case 1: ↵
                insert_rec();↵
                break;↵
            case 2: ↵
                delete_rec();↵
                break;↵
            default: ↵
                printf("Commands: 0 = Search, 1 = Insert, 2 = Delete \n");↵
                break;↵
        }↵
    }↵
}↵
```



训练原理：make与Makefile

addressbook.h 文件示例如下。↵

```
#include <stdio.h>↵
#include <string.h>↵
#include <stdlib.h>↵
↵
#define MAX_RECORD 35↵
#define MSG_LEN 20↵
char names[MAX_RECORD][MSG_LEN], student_id[MAX_RECORD][MSG_LEN], emails[MAX_
RECORD][MSG_LEN];↵
int num_record;↵
void print_onerow(int index);↵
void print_all();↵
int find_rec();↵
int insert_rec();↵
int delete_rec();↵
```



训练原理：make与Makefile

insert.c 文件示例如下。↵

```
#include "addressbook.h"↵
int insert_rec()↵
{↵
    char msg_str[MSG_LEN];↵
    int i;↵
    if (num_record == MAX_RECORD) {↵
        printf("-- No space left --\n");↵
        return 1;↵
    }↵
    printf("Enter name: ");↵
    scanf("%s", msg_str);↵
    strcpy(names[num_record], msg_str);↵
    printf("Enter student id: ");↵
    scanf("%s", msg_str);↵
    strcpy(student_id[num_record], msg_str);↵
    printf("Enter email: ");↵
    scanf("%s", msg_str);↵
    strcpy(emails[num_record], msg_str);↵
    num_record++;↵
    printf("Insert successful!\n\n");↵
    return 0;↵
}↵
```



训练原理：make与Makefile

delete.c 文件示例如下。↵

```
#include "addressbook.h"↵
int delete_rec()↵
{↵
    int i, index;↵
    if (num_record == 0){↵
        printf("No item in the address book!\n");↵
        return -1;↵
    }↵
    do{↵
        printf("Enter index of record to be deleted (0-%d): ", num_record-1);↵
        scanf("%d", &index);↵
    }while(index < 0 && index >= num_record);↵
    for(i = index; i < num_record; i++){↵
        strcpy(names[i], names[i+1]);↵
        strcpy(student_id[i], student_id[i+1]);↵
        strcpy(emails[i], emails[i+1]);↵
    }↵
    printf("Delete successful!\n\n");↵
    --num_record;↵
    return 0;↵
}↵
```



训练原理：make与Makefile

find.c 文件示例如下。↵

```
#include "addressbook.h"↵
int find_rec()↵
{↵
    char msg_str[MSG_LEN];↵
    int i;↵
    printf("Enter names or student id or email address: ");↵
    scanf("%s", msg_str);↵
    for(i = 0; i < num_record; i++){↵
        if (strcmp(names[i], msg_str) == 0 || strcmp(student_id[i], msg_str) == 0↵
        || strcmp(emails[i], msg_str) == 0)↵
            break;↵
    }↵
    if (i < num_record)↵
        print_onerow(i);↵
    else↵
        printf("No such record!\n");↵
    printf("\n");↵
    return 0;↵
}↵
```



训练原理：make与Makefile

- 分别通过gcc命令将这4个C语言源文件和一个C语言头文件编译成一个目标文件，过程如下

```
gcc -c main.c
```

```
gcc -c insert.c
```

```
gcc -c delete.c
```

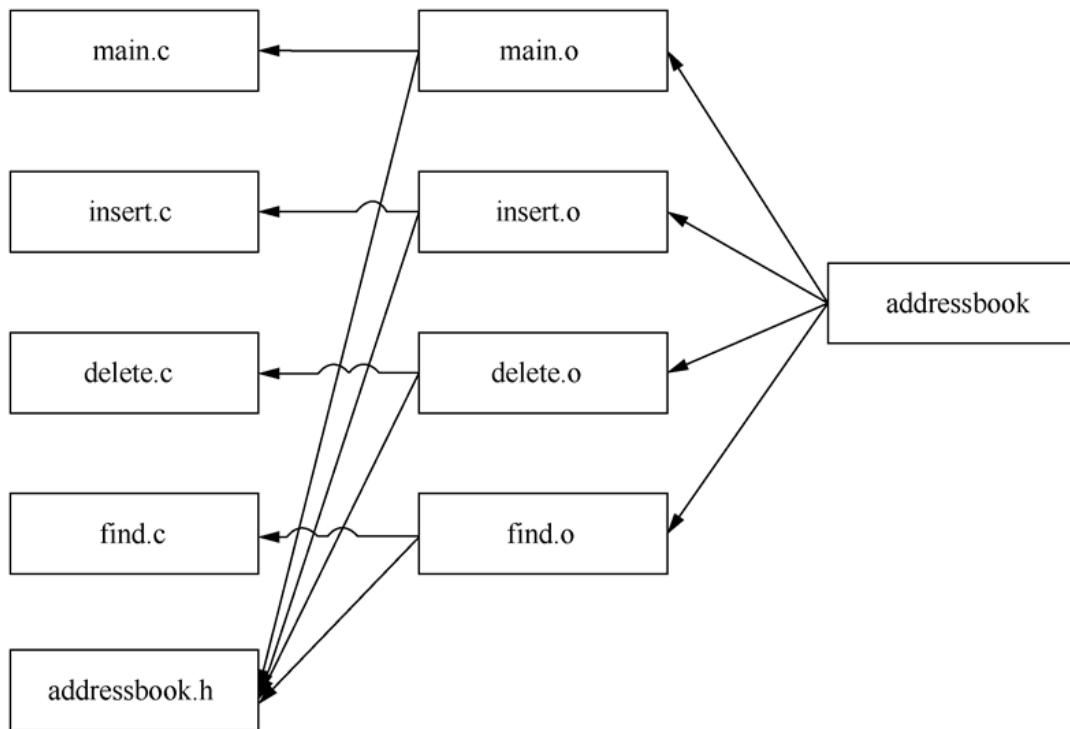
```
gcc -c find.c
```

```
gcc main.o insert.o delete.o find.o -o addressbook
```



训练原理：make与Makefile

- addressbook程序的依赖关系如下图



训练原理：make与Makefile

- 构建系统的目标是把源代码转换为可执行程序，还支持Web应用打包、文档生成、源代码自动分析以及很多其他相关活动
- 构建工具使用依赖关系图来判断要对哪些文件进行编译，以及何时进行编译
- 当源文件的规模稍微大一些，依赖关系就变得很复杂，就需要某种形式的自动构建系统
- GNU make是一个控制源代码生成可执行文件或其他文件的工具，通过名为Makefile的文件来记录源文件和目标文件之间的关系和构建规则



Makefile里主要有什么

- 显式规则、隐晦规则、变量定义、文件指示和注释

1、显式规则。显式规则说明了，如何生成一个或多的的目标文件。这是由Makefile的书写者明显指出，要生成的文件，文件的依赖文件，生成的命令。

2、隐晦规则。由于GNU make有自动推导的功能，所以隐晦的规则可以比较简略地方式书写Makefile

3. 变量的定义

在Makefile中可以定义一系列变量，类似于C语言中的宏，变量一般都是字符串。当Makefile被执行时，定义的变量都会被扩展到相应的引用位置上



Makefile里主要有什么

- 显式规则、隐晦规则、变量定义、文件指示和注释

4. 文件指示

文件指示又包括3种类型：指定一个Makefile中包含另一个Makefile，类似C语言中的`#include`命令；根据某些条件指定Makefile中的有效部分，类似C语言中的预编译语句`#if`；定义一个多行的命令。

5. 注释

Makefile中只支持行注释，其注释是用“`#`”标注。如果要在Makefile中使用“`#`”字符，必须通过反斜线进行转义，如“`\#`”。



Makefile的主要规则

target ... : prerequisites ...

command

...

- target是一个目标，可以是目标文件（main.o），也可以是执行文件（addressbook），还可以是一个标签（伪目标）。
- prerequisites是要生成那个target所需要依赖的文件或者目标。
- command也就是make需要执行的命令（可能是任意的shell命令，可能是一个或多个）



Makefile的主要规则

target ... : prerequisites ...

command

...

- 一个规则可以有多个命令行，每一条命令占一行。 注意：每一个命令行必须以[Tab]字符开始，[Tab]字符告诉 make 此行是一个命令行。make按照命令完成相应的动作。这也是书写 Makefile 中容易产生，而且比较隐蔽的错误。
- Makefile的规则是Makefile中最核心的内容



构建addressbook程序的Makefile

```
addressbook: main.o insert.o delete.o find.o                #第 1 行代码↵
    gcc -g -o addressbook main.o insert.o delete.o find.o #第 2 行代码↵
↵
main.o: main.c addressbook.h                                #第 3 行代码↵
    gcc -g -c main.c                                         #第 4 行代码↵
↵
insert.o: insert.c addressbook.h                             #第 5 行代码↵
    gcc -g -c insert.c                                       #第 6 行代码↵
↵
delete.o: delete.c addressbook.h                             #第 7 行代码↵
    gcc -g -c delete.c                                       #第 8 行代码↵
↵
find.o: find.c addressbook.h                                 #第 9 行代码↵
    gcc -g -c find.c                                         #第 10 行代码↵
```



构建addressbook程序的Makefile

- 第1行代码声明一个名为addressbook的文件，它依赖于其他所有文件：
main.o、insert.o、delete.o、find.o
- 第2行代码提供了一个Linux命令，用来根据所有这些目标文件生成
addressbook文件
- 第3行代码指出main.o同时依赖于main.c和addressbook.h两个文件
- 第4行代码则指出了用来编译main.c的Linux命令
- 后面的Makefile内容所提供的规则与上面类似，分别指定了其他源文件和目标文件。



通过make构建addressbook程序

- 当完成上述Makefile文件编写后，需要将Makefile文件与main.c, insert.c, delete.c, find.c, addressbook.h等文件放在同一目录下，然后在Linux Shell中进入该目录，执行make命令，来进行addressbook程序的构建，构建过程如下：

```
$ make↵  
gcc -g -c main.c↵  
gcc -g -c insert.c↵  
gcc -g -c delete.c↵  
gcc -g -c find.c↵  
gcc -g -o addressbook main.o insert.o delete.o find.o↵
```



通过make构建addressbook程序

- 当完成上述Makefile文件编写后，需要将Makefile文件与main.c, insert.c, delete.c, find.c, addressbook.h等文件放在同一目录下，然后在Linux Shell中进入该目录，执行make命令，来进行addressbook程序的构建，构建过程如下：

```
$ make↵  
gcc -g -c main.c↵  
gcc -g -c insert.c↵  
gcc -g -c delete.c↵  
gcc -g -c find.c↵  
gcc -g -o addressbook main.o insert.o delete.o find.o↵
```

- GNU make程序检查Makefile文件，在内存中重新生成依赖关系，然后判断要执行什么命令。因为在创建addressbook程序之前，所有目标文件必须都已存在，所以形成了以上所示的命令输出顺序。



通过make构建addressbook程序

- Makefile支持增量式构建，GNU make会判断某些文件是否真的需要编译，或这些文件是否已存在
- 在完成对addressbook程序的首次构建之后，如果没有任何修改就再次执行make命令，那么将会出现如以下示例所示的提示信息。

```
$ make↵  
make: 'addressbook' is up to date.↵
```



通过make构建addressbook程序

- 如果修改了某个源文件（例如main.c），就改变了这个文件的时间戳。这样一旦执行GNU make就会判断main.o不是新的（其依赖的main.c已更新），从而决定重新编译main.c生成main.o，然后重新生成addressbook文件

```
$make↵  
gcc -g -c main.c↵  
gcc -g -o addressbook main.o insert.o delete.o find.o↵
```



通过make构建addressbook程序

- 所有的Linux命令（第2、4、6、8、10行）必须前置一个Tab字符，而不是空格。这是历史原因造成的，也是让许多Makefile初学者疑惑的地方。如果忘记了这一点，就会产生如以下示例所示的出错信息

```
Makefile:2: ... missing separator (did you mean TAB instead of 8 spaces?) .<
```



训练二：gcc+gdb+make训练

- 训练内容

- 在Linux下通过C语言程序设计文本格式化的应用程序，要求通过不同的源文件实现不同的功能模块
- 针对多个源文件组成的C语言工程编写Makefile
- 在Linux下通过GNU工具链构建、调试、运行该应用程序



文本格式化

- 读取source里的内容，source中的单词之间存在空格、制表符、空行等，如下所示：

The Very Hungry Caterpillar by Eric Carle In the light

of the moon a little egg lay on a leaf.

One Sunday morning the warm sun came up....and POP,

out of the egg came a tiny, very hungry caterpillar. He started

looking for some food.



文本格式化

编写程序，对源文件进行如下操作：

- 1) 删除额外的空格和空行
- 2) 对每行文字进行填充和调整。
 - 填充行：添加单词直到再多加一个单词导致超出该行最大限制
 - 调整行：单词间添加额外的空格以便每行长度相同
- 3) 最后一行不调整
- 编译生成执行程序fmt，并在命令行中执行以下命令，将结果打印到newfile中：
- `fmt < sourcefile > newfile`



文本格式化

- newfile中的内容如下:
- The Very Hungry Caterpillar by Eric Carle In the light of the moon a little egg lay on a leaf. One Sunday morning the warm sun came up....and POP, out of the egg came a tiny, very hungry caterpillar. He started looking for some food.



文本格式化

- 要求程序按以下文件进行组织:

word.h //read_word函数定义

word.c //read_word函数的实现, 读取一个单词

line.h //包含所有与行缓冲区相关的函数

line.c //所有与行缓冲区相关的函数实现

fmt.c //包含main函数

- 编写以上文件及makefile



• 主函数示例如下：

```
for(;;){  
    读单词;  
    if(不能读单词){  
        输出行缓冲区的内容，不进行调整;  
        终止程序;  
    }  
    if(行缓冲区已经填满){  
        输出行缓冲区内容，进行调整;  
        清除行缓冲区;  
    }  
    往行缓冲区中添加单词;  
}
```

